```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)

print("All libraries imported successfully!")
```

```
All libraries imported successfully!
```

```
# Cell 2: Download the SMS Spam Collection Dataset

import urllib.request
import zipfile
import os

url = "https://archive.ics.uci.edu/static/public/228/sms+spam+collection.zip"

urllib.request.urlretrieve(url, "sms_spam_collection.zip")

with zipfile.ZipFile("sms_spam_collection.zip", "r") as zip_ref:
    zip_ref.extractall(".")

print("SMS Spam Collection dataset downloaded successfully!")
print("Dataset file exists:", os.path.exists("SMSSpamCollection"))
```

```
SMS Spam Collection dataset downloaded successfully!
Dataset file exists: True
```

```
# Cell 3: Load the SMS Spam Dataset

df = pd.read_csv(
    "SMSSpamCollection",
    sep="\t",
    header=None,
    names=["label", "message"]
)

print("Dataset loaded successfully!")

df.head()
```

```
Dataset loaded successfully!
```

	label	message
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

Next steps: Generate code with df | New interactive sheet

```
# Cell 4: Explore Dataset Information

print("Total number of messages:", df.shape[0])

print("\nDataset dimensions:", df.shape)

print("\nMessage distribution:")
print(df["label"].value_counts())
```

```
print("\nMissing values:")
print(df.isnull().sum())
```

```
Total number of messages: 5572

Dataset dimensions: (5572, 2)

Message distribution:
label
ham     4825
spam     747
Name: count, dtype: int64

Missing values:
label      0
message    0
dtype: int64
```

```
# Cell 5: Convert Text Labels into Numerical Values

df["label_num"] = df["label"].map({
    "ham": 0,
    "spam": 1
})

print("Labels converted successfully!")

df.head()
```

```
Labels converted successfully!
```

	label	message	label_num
0	ham	Go until jurong point, crazy.. Available only ...	0
1	ham	Ok lar... Joking wif u oni...	0
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	1
3	ham	U dun say so early hor... U c already then say...	0
4	ham	Nah I don't think he goes to usf, he lives aro...	0

Next steps: Generate code with df | New interactive sheet

```
# Cell 6: Separate Features and Target Labels

X = df["message"]
y = df["label_num"]

print("Input data (X) prepared successfully!")
print("Target labels (y) prepared successfully!")

print("\nNumber of input messages:", len(X))
print("Number of target labels:", len(y))
```

```
Input data (X) prepared successfully!
Target labels (y) prepared successfully!

Number of input messages: 5572
Number of target labels: 5572
```

```
# Cell 7: Split Dataset into Training and Testing Sets

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

print("\nTraining and testing data prepared successfully!")
```

```
Training samples: 4457
Testing samples: 1115

Training and testing data prepared successfully!
```

```
# Cell 8: Convert Text Messages into Numerical Features using TF-IDF

vectorizer = TfidfVectorizer(
    lowercase=True,
    stop_words="english",
    max_features=5000
)

X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

print("TF-IDF vectorization completed successfully!")

print("\nTraining matrix shape:", X_train_tfidf.shape)
print("Testing matrix shape:", X_test_tfidf.shape)
```

```
TF-IDF vectorization completed successfully!

Training matrix shape: (4457, 5000)
Testing matrix shape: (1115, 5000)
```

```
# Cell 9: Train the Multinomial Naive Bayes Model

model = MultinomialNB(alpha=0.5)

model.fit(X_train_tfidf, y_train)

print("Multinomial Naive Bayes model trained successfully!")
print("Model is ready for spam message classification.")
```

```
Multinomial Naive Bayes model trained successfully!
Model is ready for spam message classification.
```

```
# Cell 10: Predict Spam Labels on Test Data

y_pred = model.predict(X_test_tfidf)

print("Predictions generated successfully!")

print("\nFirst 20 predicted labels:")
print(y_pred[:20])

print("\nFirst 20 actual labels:")
print(y_test.iloc[:20].values)
```

```
Predictions generated successfully!

First 20 predicted labels:
[0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]

First 20 actual labels:
[0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
```

```
# Cell 11: Calculate Model Accuracy

accuracy = accuracy_score(y_test, y_pred)

print("Model Accuracy:", accuracy)
print("Accuracy Percentage:", accuracy * 100)
```

```
Model Accuracy: 0.979372197309417
Accuracy Percentage: 97.9372197309417
```

```
# Cell 12: Generate Classification Report

print("Classification Report:\n")

print(classification_report(
    y_test,
    y_pred,
    target_names=["Ham", "Spam"]
))
```

```
Classification Report:

              precision    recall  f1-score   support

         Ham       0.98      1.00      0.99       966
```

```
           Spam       1.00      0.85      0.92       149

       accuracy                           0.98      1115
      macro avg       0.99      0.92      0.95      1115
   weighted avg       0.98      0.98      0.98      1115
```

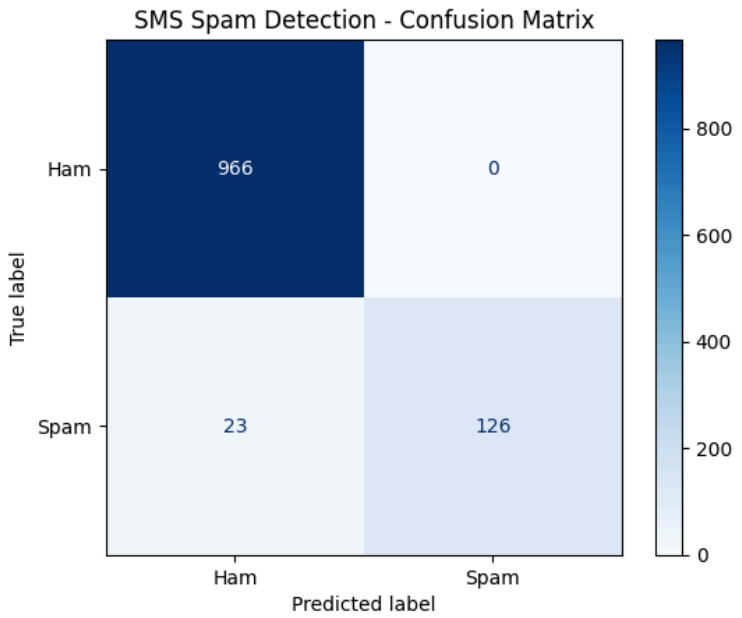
```
# Cell 13: Display Confusion Matrix

cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Ham", "Spam"]
)

disp.plot(cmap="Blues", values_format="d")

plt.title("SMS Spam Detection - Confusion Matrix")
plt.show()
```




```
# Cell 14: Create Spam Prediction Function

def predict_spam(message):

    message_tfidf = vectorizer.transform([message])

    prediction = model.predict(message_tfidf)[0]

    if prediction == 1:
        return "SPAM"
    else:
        return "NOT SPAM"
```

```
# Cell 15: Test a Spam Message

message = "Congratulations! You have won a free prize. Call now!"

print(predict_spam(message))
```

```
SPAM
```

```
# Cell 16: Test a Normal Message

message = "Hey, are we meeting at 6 pm today?"

print(predict_spam(message))
```

```
NOT SPAM
```

```
# Cell 17: Create Spam Probability Function
```

```
def predict_spam_with_probability(message):

    message_tfidf = vectorizer.transform([message])

    prediction = model.predict(message_tfidf)[0]

    probabilities = model.predict_proba(message_tfidf)[0]

    spam_probability = probabilities[1] * 100

    if prediction == 1:
        result = "SPAM"
    else:
        result = "NOT SPAM"

    print("Message:", message)
    print("Prediction:", result)
    print(f"Spam Probability: {spam_probability:.2f}%")
```

```
# Cell 18: Test Spam Probability Prediction

predict_spam_with_probability(
    "URGENT! You have won a free lottery prize. Claim now!"
)
```

```
Message: URGENT! You have won a free lottery prize. Claim now!
Prediction: SPAM
Spam Probability: 99.18%
```

```
# Additional Visualization: Dataset Distribution

plt.figure(figsize=(6, 4))

label_counts = df["label"].value_counts().reindex(["ham", "spam"])

plt.bar(label_counts.index, label_counts.values)

plt.title("Distribution of Ham and Spam Messages")
plt.xlabel("Message Category")
plt.ylabel("Number of Messages")

plt.show()
```

